Technical Implementation Report: Prompt- and Shift-Aware Conformal Reliability of Zero-Shot Vision-Language Models

Software Documentation Maintainer

October 2026

Abstract

This report documents the software architecture, data flow, mathematical operations, configuration parameters, and verification procedures implemented in `prompt_shift_conformal_vlm_persistent.py` and its synchronized Jupyter notebook `prompt_shift_conformal_vlm_persistent.ipynb`. The codebase implements a reproducible evaluation pipeline for Google Colab (with persistent Google Drive storage and a local filesystem fallback) that applies split-conformal prediction to a frozen OpenAI CLIP `ViT-B/32` zero-shot classifier across CIFAR-10 and CIFAR-10-C. The implementation covers resumable dataset acquisition and integrity verification, stratified index-split generation across five random seeds, cached L2-normalized text and image embedding extraction, prompt-template and ensemble logit computation, L-BFGS post-hoc scalar temperature scaling, deterministic Adaptive Prediction Sets (APS) and Regularized Adaptive Prediction Sets (RAPS), per-condition reliability metric calculation, across-seed percentile bootstrap aggregation, programmatic table and figure generation, automated forensic auditing, and final archive packaging. In accordance with its technical documentation scope, this report explains how the software is constructed and how its components interact without presenting experimental measurements or performance claims.

Contents

1 Introduction and Scope **2**

2 Project Structure and Software Dependencies **3**

2.1 Delivered Source Files 3

2.2 Runtime Directory Hierarchy 3

2.3 Software Dependencies 3

3 Overall Implementation Workflow **5**

4 Input Data and Preparation **6**

4.1 CIFAR-10 Acquisition and Verification 6

4.2 CIFAR-10-C Archive Acquisition, Extraction, and Alignment 6

4.3 Stratified Split Construction 6

5 Main Components **7**
5.1 Filesystem, Hashing, and Cache Validation Helpers (Section 3) 7
5.2 Dataset Wrappers (Sections 4 and 6) 8
5.3 Embedding and Logit Caching Layer (Sections 5–8) 8

6 Model and Algorithm Implementation **8**
6.1 Zero-Shot CLIP Similarity Logits and Prompt Ensembling 8
6.2 Constrained Post-Hoc Scalar Temperature Scaling 9
6.3 Deterministic Split-Conformal Prediction (APS and RAPS) 9

7 Processing and Experiment Execution Procedure **11**
7.1 Experiment Matrix Construction 11
7.2 Resumable Execution Loop 11

8 Configuration and Parameter Handling **11**

9 Evaluation, Aggregation, and Postprocessing Implementation **13**
9.1 Per-Condition Evaluation Metrics 13
9.2 Selective Top-1 Risk-Coverage Curves 13
9.3 Seed-Level Statistical Aggregation 14

10 Output Generation and File Handling **14**

11 Error Handling, Reproducibility, and Execution Considerations **15**
11.1 Randomness Control and Determinism 15
11.2 Exception Handling and Forensic Auditing 15

12 Implementation Limitations **16**

13 Summary of the Implementation **16**

A Function and Class Interface Reference **16**

B Notebook-to-Script Cell Mapping **19**

C Summary of Code Cleanup and Audit Findings **20**

1 Introduction and Scope

The supplied project evaluates how split-conformal prediction sets constructed around a frozen zero-shot vision-language model behave when the test distribution differs from the calibration distribution. Specifically, the software isolates two axes of distribution shift on 10-class image classification:

1. **Prompt shift:** changing the natural-language class template used to form zero-shot text embeddings at evaluation time relative to the template used at calibration time, or ensembling logits across multiple templates.
2. **Visual shift:** evaluating a classifier calibrated on clean CIFAR-10 training images against corrupted CIFAR-10-C test images across multiple corruption families and severity levels, and comparing that baseline against a matched-shift calibration regime in which disjoint calibration and evaluation subsets are drawn from the same corrupted condition.

The implementation is intentionally scoped as a controlled empirical stress-test pipeline rather than a model-training framework or a novel conformal algorithm. The vision-language backbone (OpenAI CLIP `ViT-B/32`) is kept strictly frozen with gradients disabled across all parameters; no fine-tuning, prompt tuning, or linear probing is implemented. Post-hoc adaptation is restricted to scalar temperature scaling fitted by maximum likelihood on the active calibration split prior to conformal score computation.

All functionality exists in two synchronized representations:

- `prompt_shift_conformal_vlm_persistent.ipynb`, a 43-cell Jupyter notebook (22 code cells and 21 Markdown cells) structured for sequential execution in Google Colab.
- `prompt_shift_conformal_vlm_persistent.py`, a standalone Python script containing the exact code cells and section docstrings of the notebook.

Alongside the reviewed source files, the project audit includes `code_changes.py`, which records all substantive cleanups and bug fixes applied during code review.

2 Project Structure and Software Dependencies

2.1 Delivered Source Files

The reviewed codebase consists of two executable source files (`prompt_shift_conformal_vlm_persistent.py` and `prompt_shift_conformal_vlm_persistent.ipynb`), the change-audit module (`code_changes.py`), `README.md`, and this technical report (`report.tex` and `report.pdf`). No separate local helper packages are imported by the script; all classes, helper functions, and execution loops reside within the main module and notebook.

2.2 Runtime Directory Hierarchy

At runtime, Section 2 of the code initializes a self-contained directory tree rooted at `ROOT`. When `USE_GOOGLE_DRIVE` is `True` (the default), `ROOT` resolves to `/content/drive/MyDrive/prompt_shift_conformal_vlm`; when `False`, it resolves to `/content/prompt_shift_conformal_vlm`. Table 1 lists the directory path variables created at startup and their roles.

2.3 Software Dependencies

The implementation uses three categories of libraries:

1. **Python standard library:** `__future__`, `dataclasses` (`dataclass`, `field`), `hashlib`, `importlib`, `json`, `math`, `pathlib` (`Path`), `platform`, `random`, `shutil`, `subprocess`, `sys`, `tarfile`, `time`, `typing`, `urllib.error`, `urllib.parse`, `urllib.request`, `warnings`, and `zipfile`.

Table 1: Workspace directory variables defined in Section 2 and their runtime responsibilities.

Variable	Relative Path	Responsibility
ROOT	.	Top-level persistent project directory; also stores generated README.md, EXPERIMENT_PROTOCOL.md, LIMITATIONS.md, requirements.txt, and the final ZIP archive.
DATA_ROOT	data/	Parent directory for dataset archives and extracted dataset folders, including CIFAR-10-C.tar (CIFAR10C_TAR).
CIFAR10_ROOT	data/cifar10/	Storage location for the official torchvision.datasets.CIFAR10 train and test partitions.
CIFAR10C_ROOT	data/cifar10c/	Destination for extracted CIFAR-10-C corruption arrays (<corruption>.npy) and labels.npy.
SPLITS_ROOT	splits/	Stores compressed .npz index-split files for the fixed shifted test set, clean calibration splits, and matched-shift splits.
CACHE_ROOT	cache/	Parent directory for persistent model checkpoints, text embeddings, image embeddings, logits, and calibration states.
CLIP_WEIGHTS_ROOT	cache/clip_weights/	Stores the SHA-256-verified ViT-B-32.pt checkpoint (CLIP_WEIGHTS_PATH).
CLEAN_EMBED_ROOT	cache/image_embeddings/	Stores clean_test_all.npz and clean_cal_seed_<seed>.npz.
SHIFT_EMBED_ROOT	cache/cifar10c_embeddings/	Stores per-corruption, per-severity .npz image embedding caches.
LOGIT_CACHE_ROOT	cache/logits/	Stores cached [N, 10] logit matrices organized by condition tag and prompt name.
CAL_STATE_ROOT	cache/calibration_states/	Stores per-condition fitted temperature and conformal quantile JSON files.
RESULTS_ROOT	results/	Stores experiment_manifest.csv, results_checkpoint.csv, results.csv, results_aggregated_across_seeds.csv, and risk_coverage.csv.
FIGURES_ROOT	results/figures/	Stores the six diagnostic PNG figures.
TABLES_ROOT	results/tables/	Stores main_comparison_table.{csv,html,png} and ablation_summary_APS.csv.
METADATA_ROOT	metadata/	Stores environment.json, config.json, forensic_audit.json, and persistence_status.json.
LOG_ROOT	logs/	Reserved log directory copied into final_artifacts/metadata/ if populated.
ASSETS_ROOT	assets/	Stores pipeline_diagram.png.
FINAL_ROOT	final_artifacts/	Staging directory assembled in Section 21 prior to creating prompt_shift_conformal_vlm_final_artifacts.zip.

2. **Pre-installed scientific and deep-learning stack:** `numpy`, `pandas`, `torch`, `torch.nn.functional`, `torch.utils.data` (`Dataset`, `DataLoader`), `torchvision`, `torchvision.datasets` (`CIFAR10`), `PIL.Image`, and `matplotlib` (`pyplot`, `colors.Normalize`).

3. **Research-specific packages installed in Section 1 (`REQUIRED_PIP`):** `clip` (installed from `git+https://github.com/openai/CLIP.git`), `ftfy`, `regex`, and `tqdm` (`tqdm.auto.tqdm`). In Google Colab environments, `google.colab.drive` and `google.colab.files` are also imported conditionally.

3 Overall Implementation Workflow

Execution proceeds sequentially through 21 numbered sections. Figure-level and table-level outputs depend strictly on files persisted by earlier stages:

1. **Stage 1 – Environment & Configuration (Sections 1–3):** Installs `REQUIRED_PIP`, imports all modules, defines protocol constants, mounts Google Drive when `USE_GOOGLE_DRIVE` is enabled, creates directory trees, seeds random number generators via `set_global_seed(PRIMARY_SEED)`, writes `environment.json` and `config.json`, and defines hashing, JSON, seed, and split-validation helper functions.

2. **Stage 2 – Dataset & Split Preparation (Section 4):** Loads CIFAR-10 train (50,000) and test (10,000) partitions; verifies or downloads `CIFAR-10-C.tar`; extracts the six primary corruption `.npy` arrays and `labels.npy`; verifies label equality across all five 10,000-image severity blocks; generates or validates the fixed 5,000-image shifted test split (`shift_test_indices`), five 5,000-image clean calibration splits (`clean_calibration_indices`), and five disjoint (2,000, 5,000) matched-shift splits (`matched_splits`); and runs explicit class-balance assertions.

3. **Stage 3 – Backbone & Feature Caching (Sections 5–7):** Verifies or downloads `ViT-B-32.pt`; loads the frozen CLIP model in evaluation mode; extracts `logit_scale`; encodes and caches the $(10, 512)$ L2-normalized class text embeddings for each of the four prompt templates in `text_features.npz`; encodes and caches the $(10000, 512)$ clean test image embeddings and five $(5000, 512)$ clean calibration image embeddings; and defines `get_shift_embeddings` for lazy extraction of corrupted image subsets.

4. **Stage 4 – Scoring, Calibration, & Sanity Testing (Sections 8–13):** Defines matrix-multiplication logit computation, arithmetic-mean logit ensembling, softmax probability conversion, L-BFGS scalar temperature scaling, APS/RAPS calibration scoring and prediction-set construction, evaluation metrics, and the 2,235-row experiment matrix (`experiment_manifest.csv`), followed by an end-to-end sanity check on 128 calibration and 256 test examples.

5. **Stage 5 – Full Experiment Execution & Aggregation (Sections 14–15):** Loops over all five seeds, 149 `ExperimentSpec` instances per seed, and three methods (`APS`, `TS+APS`, `RAPS`). Each completed row is appended to `results_checkpoint.csv`; once all 2,235 rows are present, the sorted table is written to `results.csv` and aggregated across seeds into `results_aggregated_across_seeds.csv`.

6. **Stage 6 – Postprocessing, Auditing, & Packaging (Sections 16–21):** Generates Figures 1–6 and `risk_coverage.csv`, builds `main_comparison_table.{csv,html,png}` and `ablation_summary_APS.csv`, runs `run_forensic_audit`, writes Markdown documentation files and `pipeline_diagram.png`, copies all verified deliverables into `final_`

artifacts/, writes artifact_manifest.csv, and builds prompt_shift_conformal_vlm_final_artifacts.zip.

4 Input Data and Preparation

4.1 CIFAR-10 Acquisition and Verification

The function ensure_cifar10() calls _load_or_download_cifar10_split(train=True) and _load_or_download_cifar10_split(train=False). Each call first attempts CIFAR10(root=str(CIFAR10_ROOT), train=train, download=False) so existing persistent files on Google Drive are reused without network traffic. If loading fails and AUTO_DOWNLOAD_MISSING is True, it retries with download=True. Afterwards, ensure_cifar10() asserts that list(train.classes) == CLASS_NAMES, list(test.classes) == CLASS_NAMES, len(train) == 50_000, and len(test) == 10_000, where CLASS_NAMES is the canonical 10-element list ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]. The integer target lists train_ds.targets and clean_test_ds.targets are converted to 1D int64 NumPy arrays train_labels and clean_test_labels.

4.2 CIFAR-10-C Archive Acquisition, Extraction, and Alignment

Rather than downloading the 2.92 GB Zenodo archive unconditionally, extract_selected_cifar10c_members() first calls _all_required_cifar10c_files_present(), which invokes _valid_extracted_cifar10c_file(stem) for each of the six configured corruption keys (gaussian_noise, motion_blur, fog, brightness, contrast, jpeg_compression) plus labels. A corruption file is considered valid if and only if its file size exceeds 128 bytes and np.load(path, mmap_mode="r", allow_pickle=False) returns an array with shape (50000, 32, 32, 3) and dtype uint8. The labels.npy file is valid if its shape is (50000,), its dtype is an integer type (np.issubdtype(arr.dtype, np.integer)), and all values lie in $[0, 9]$.

If any required .npy file is missing or invalid, ensure_cifar10c_archive() checks whether CIFAR-10-C.tar exists. If absent or incomplete, download_with_progress() streams the archive in 8 MiB chunks into CIFAR-10-C.tar.partial. If a partial file exists from an interrupted session, the downloader sends an HTTP Range: bytes=<resume_from>- header; if the server responds with HTTP 206 and a Content-Range header, downloading resumes in "ab" mode, whereas if the server ignores the range request, the partial file is unlinked and the download restarts cleanly from byte 0 with exponential backoff up to max_retries = 5. Once downloaded, _verify_cifar10c_archive() verifies both the MD5 digest (56bf5dcef84df0e2308c6dcbcbbd8499) and the SHA-256 digest (c72763e1...cf3cb876a7).

During extraction, extract_selected_cifar10c_members() opens CIFAR-10-C.tar via tarfile.open(archive, mode="r:*"), locates each member via _find_tar_member(), streams it in 8 MiB chunks to a .partial file, atomically renames it to data/cifar10c/<stem>.npy, and re-runs _valid_extracted_cifar10c_file(stem). Next, verify_cifar10c_alignment() verifies that for every severity level $s \in \{1, 2, 3, 4, 5\}$, the slice labels[(s - 1) * 10000 : s * 10000] is elementwise identical to clean_test_labels, and checks the leading 10 images of every severity block across all six corruptions.

4.3 Stratified Split Construction

All data splits are constructed via stratified_indices(labels, size, seed) or class-wise partitioned sampling, and persisted as compressed .npz files:

- **Fixed shifted evaluation subset (`shift_test_5000_seed_20270317.npz`):** Samples 5,000/10 = 500 indices per class without replacement from `clean_test_labels` using `np.random.default_rng(20270317)`, concatenates the per-class arrays, and shuffles the combined array in place. This single 5,000-image subset is shared across all corruptions, severities, and seeds for clean-calibrated shifted evaluations (E2–E4 and E6).

- **Clean calibration splits (`clean_calibration_seed_<seed>.npz`):** For each $s \in \{13, 42, 123, 456, 765\}$, samples 5,000/10 = 500 training indices per class without replacement from `train_labels` using `np.random.default_rng(s)`.

- **Matched-shift splits (`matched_shift_seed_<seed>.npz`):** For each $s \in \{13, 42, 123, 456, 765\}$, derives a seed via `stable_condition_seed(MATCHED_BASE_SEED, s)`:

$$\text{seed_value}(s) = \left((914{,}159 \times 1{,}000{,}003) \oplus (s \times 97{,}531) \oplus \text{0x9E3779B9}\right) \bmod (2^{32} - 1). \quad (1)$$

  Using `np.random.default_rng(seed_value)`, the code iterates over each class $c \in \{0, \ldots, 9\}$ in `clean_test_labels` (which has 1,000 images per class), samples $200 + 500 = 700$ indices without replacement, assigns the first 200 to `cal_parts` and the remaining 500 to `test_parts`, concatenates across classes, and shuffles `cal_idx` (2,000 images) and `test_idx` (5,000 images) independently. This guarantees exact class balance in both partitions and zero index overlap ($\text{cal_idx} \cap \text{test_idx} = \emptyset$).

5 Main Components

5.1 Filesystem, Hashing, and Cache Validation Helpers (Section 3)

- `sha256_file(path, chunk_size=8*1024*1024)` and `md5_file(path, chunk_size=8*1024*1024)`: Stream binary files in 8 MiB chunks through `hashlib.sha256()` or `hashlib.md5()` and return the lowercase hexadecimal digest string.

- `sanitize_name(value)`: Converts an arbitrary value to a string and replaces any character outside `[A-Za-z0-9_-]` with `"_"`.

- `atomic_write_json(path, payload)`: Writes formatted JSON (`indent=2, sort_keys=True, default=str`) to `path.with_suffix(path.suffix + ".tmp")` and atomically replaces `path` via `Path.replace`.

- `save_indices(path, indices, labels, metadata)` and `load_valid_indices(path, expected_size, reference_labels=None)`: Persist and validate 1D integer index arrays alongside their corresponding label slices and JSON metadata. `load_valid_indices` returns `None` (triggering regeneration) if the file is missing, unreadable, not 1D, has length or unique-element count different from `expected_size`, contains out-of-bounds indices, or has stored labels that disagree with `reference_labels[indices]`.

- `finite_and_normalized_probabilities(probs, atol=1e-5)`: Raises an `AssertionError` if `probs.shape` is not `(N, NUM_CLASSES)`, or a `FloatingPointError` if any entry is non-finite, outside $[-\text{atol}, 1 + \text{atol}]$, or if row sums deviate from 1.0 by more than `atol`.

- `cache_npz_valid(path, required_arrays)`: Returns `True` only if `path` exists, exceeds 128 bytes, can be opened with `np.load(..., allow_pickle=False)`, and contains all keys in `required_arrays`.

5.2 Dataset Wrappers (Sections 4 and 6)

- `CIFAR10CSubset(corruption_path, labels_path, severity, sample_indices, preprocess)`: Opens the corruption `.npy` array and `labels.npy` as read-only memory-mapped arrays (`mmap_mode="r"`), validates shapes and dtypes, sets `self.base_offset = (severity - 1) * 10_000`, and on `__getitem__(i)` reads `idx = int(self.sample_indices[i])`, slices `self.images[self.base_offset + idx]`, converts the $(32, 32, 3)$ `uint8` array to a PIL image via `Image.fromarray(image_array)`, applies the CLIP `preprocess` transform, and returns `(tensor, label, idx)`.
- `CleanCIFARSubset(base_dataset, indices, preprocess_fn)`: Wraps a `torchvision.datasets.CIFAR10` instance and index array `indices`, returning `(preprocess_fn(image), int(label), idx)` for each item so `encode_image_embeddings` can verify sample ordering.

5.3 Embedding and Logit Caching Layer (Sections 5–8)

- `ensure_clip_weights()`, `build_text_features()`, and `load_or_build_text_features()`: Manage the persistent CLIP checkpoint and the $(10, 512)$ normalized text embedding dictionary `TEXT_FEATURES`.
- `encode_image_embeddings(dataset, cache_path, expected_size, expected_ids=None, expected_labels=None, batch_size=None)`: Checks whether `cache_path` already holds a valid $(N, 512)$ float32 matrix whose L2 row norms are within 2×10^{-4} of 1.0 and whose stored `sample_ids` and `labels` match `expected_ids` and `expected_labels`. On a cache miss, it runs `model.encode_image(batch_images).float()` inside `torch.inference_mode()`, normalizes along `dim=-1`, validates finiteness and unit norms, and saves the result to `cache_path`.
- `get_shift_embeddings(corruption, severity, sample_indices, tag)`: Wraps `CIFAR10CSubset` and delegates to `encode_image_embeddings` with cache path `cache/cifar10c_embeddings/<corruption>/severity_<severity>/<tag>.npz`.
- `logits_from_embeddings(image_embeddings, prompt_condition)` and `logits_cached(...)`: Compute and persist $(N, 10)$ float32 similarity logits.

6 Model and Algorithm Implementation

6.1 Zero-Shot CLIP Similarity Logits and Prompt Ensembling

Given an image x, the frozen CLIP image encoder produces a feature vector $f_{\text{img}}(x) \in \mathbb{R}^{512}$, which is L2-normalized to unit length:

$$\hat{u}(x) = \frac{f_{\text{img}}(x)}{\|f_{\text{img}}(x)\|_2}. \tag{2}$$

For each prompt template p in {`P0_canonical`, `P1_paraphrase`, `P2_generic_visual`, `P3_photo_wording`} and class name c_j ($j \in \{0, \ldots, 9\}$), the tokenized string is encoded by `model.encode_text` into $f_{\text{txt}}(p, c_j) \in \mathbb{R}^{512}$ and L2-normalized:

$$\hat{v}_{p,j} = \frac{f_{\text{txt}}(p, c_j)}{\|f_{\text{txt}}(p, c_j)\|_2}. \tag{3}$$

Let $V_p \in \mathbb{R}^{10\times 512}$ denote the matrix whose j-th row is $\hat{v}_{p,j}^\top$, and let $s = \exp(\texttt{model.logit_scale}) \in \mathbb{R}_{>0}$ denote the frozen scalar logit scale ($s = 100.0$ in the `ViT-B/32` checkpoint). In `logits_from_embeddings`, the zero-shot logit vector $z_p(x) \in \mathbb{R}^{10}$ for a single prompt template p is computed as:

$$z_p(x) = s \cdot V_p \hat{u}(x). \tag{4}$$

When `prompt_condition` equals `PROMPT_ENSEMBLE` (`"ensemble_mean_logits"`), `logits_from_embeddings` recursively computes $z_p(x)$ for all four templates in `PROMPTS` and takes their arithmetic mean in logit space:

$$z_{\text{ens}}(x) = \frac{1}{|\mathcal{P}|} \sum_{p\in\mathcal{P}} z_p(x), \quad |\mathcal{P}| = 4. \tag{5}$$

6.2 Constrained Post-Hoc Scalar Temperature Scaling

In `probability_from_logits(logits, temperature=1.0)`, a logit vector $z \in \mathbb{R}^{10}$ is mapped to a categorical probability vector $\hat{\pi}(z;T) \in \Delta^9$ via temperature-scaled softmax in `float64` precision:

$$\hat{\pi}_j(z;T) = \frac{\exp(z_j/T)}{\sum_{l=0}^{9}\exp(z_l/T)}, \quad j \in \{0,\dots,9\}. \tag{6}$$

For the `APS` and `RAPS` methods, T is fixed at 1.0. For `TS+APS`, `fit_temperature(logits, labels)` fits T on the active calibration split $\{(z_i, y_i)\}_{i=1}^{n_{\text{cal}}}$ by minimizing the empirical cross-entropy loss. To guarantee $T \in [T_{\min}, T_{\max}] = [0.05, 20.0]$ throughout optimization without unconstrained line-search excursions, T is parameterized by an unconstrained scalar $w \in \mathbb{R}$ via a scaled logistic sigmoid:

$$T(w) = T_{\min} + (T_{\max} - T_{\min})\,\sigma(w), \quad \sigma(w) = \frac{1}{1+e^{-w}}. \tag{7}$$

The parameter w is initialized at the geometric midpoint $T_0 = \sqrt{T_{\min}T_{\max}} = 1.0$ via `_inv_sigmoid`:

$$w_0 = \log\left(\frac{f_0}{1-f_0}\right), \quad f_0 = \text{clip}\left(\frac{\sqrt{T_{\min}T_{\max}} - T_{\min}}{T_{\max} - T_{\min}}, 10^{-6}, 1-10^{-6}\right), \tag{8}$$

and optimized with `torch.optim.LBFGS` (`lr=0.5`, `max_iter=50`, `tolerance_grad=1e-8`, `tolerance_change=1e-8`, `line_search_fn="strong_wolfe"`) over the objective:

$$\mathcal{L}(w) = -\frac{1}{n_{\text{cal}}} \sum_{i=1}^{n_{\text{cal}}} \log \hat{\pi}_{y_i}(z_i; T(w)). \tag{9}$$

6.3 Deterministic Split-Conformal Prediction (APS and RAPS)

Given a probability vector $\hat{\pi}(x) \in \Delta^9$, `sorted_probabilities(probs)` sorts the class probabilities in descending order using a stable sort (`np.argsort(-probs, axis=1, kind="stable")`). Let $\pi_{(1)}(x) \geq \pi_{(2)}(x) \geq \cdots \geq \pi_{(10)}(x)$ denote the sorted probabilities, let $C_r(x) = \sum_{j=1}^{r} \pi_{(j)}(x)$ denote the cumulative probability up to 1-based rank $r \in \{1,\dots,10\}$, and let $r(x,y) \in \{1,\dots,10\}$ denote the 1-based rank of class y in the descending ordering (represented in code as `rank0 + 1`, where `rank0 = ranks[i, y]` is 0-based).

- **Deterministic APS score (`aps_calibration_scores`):** The nonconformity score $S_{\mathrm{APS}}(x, y)$ is the cumulative probability mass required to include the true label y:

$$S_{\mathrm{APS}}(x, y) = C_{r(x,y)}(x) = \sum_{j=1}^{r(x,y)} \pi_{(j)}(x). \tag{10}$$

- **Deterministic RAPS score (`raps_calibration_scores`):** Adds a linear rank penalty with regularization weight $\lambda = \texttt{RAPS_LAMBDA} = 0.01$ for ranks exceeding $k_{\mathrm{reg}} = \texttt{RAPS_K_REG} = 5$:

$$S_{\mathrm{RAPS}}(x, y) = C_{r(x,y)}(x) + \lambda \max(r(x, y) - k_{\mathrm{reg}}, 0). \tag{11}$$

- **Finite-sample order-statistic quantile (`conformal_quantile`):** Given calibration scores $s_1, \ldots, s_{n_{\mathrm{cal}}}$ and miscoverage level $\alpha = 0.10$, the order-statistic index k is computed directly via `math.ceil`:

$$k = \min\Big(\max\big(\lceil (n_{\mathrm{cal}} + 1)(1 - \alpha) \rceil,\, 1\big),\, n_{\mathrm{cal}}\Big), \quad \hat{q} = s_{(k)}, \tag{12}$$

  where $s_{(1)} \le s_{(2)} \le \cdots \le s_{(n_{\mathrm{cal}})}$ are the calibration scores sorted via mergesort. For clean calibration ($n_{\mathrm{cal}} = 5{,}000, \alpha = 0.10$), $k = \lceil 5001 \times 0.90 \rceil = \lceil 4500.9 \rceil = 4501$; for matched-shift calibration ($n_{\mathrm{cal}} = 2{,}000, \alpha = 0.10$), $k = \lceil 2001 \times 0.90 \rceil = \lceil 1800.9 \rceil = 1801$.

- **Prediction-set construction (`prediction_sets_aps` and `prediction_sets_raps`):** For a test input x, the boolean prediction set $\widehat{\mathcal{C}}(x) \subseteq \{0, \ldots, 9\}$ includes all classes whose score does not exceed $\hat{q}$ plus a numerical tolerance of 10^{-12}:

$$\widehat{\mathcal{C}}_{\mathrm{APS}}(x) = \{y \in \{0, \ldots, 9\} : C_{r(x,y)}(x) \le \hat{q} + 10^{-12}\}, \tag{13}$$

$$\widehat{\mathcal{C}}_{\mathrm{RAPS}}(x) = \{y \in \{0, \ldots, 9\} : C_{r(x,y)}(x) + \lambda \max(r(x, y) - k_{\mathrm{reg}}, 0) \le \hat{q} + 10^{-12}\}. \tag{14}$$

Listing 1: Core conformal quantile and deterministic APS/RAPS prediction-set construction in Section 10.

```
def conformal_quantile(scores: np.ndarray, alpha: float) -> float:
    scores = np.asarray(scores, dtype=np.float64)
    if scores.ndim != 1 or len(scores) == 0:
        raise ValueError("Conformal scores must be a non-empty 1D array.")
    if not np.isfinite(scores).all():
        raise FloatingPointError("Non-finite calibration scores.")
    n = len(scores)
    k = int(math.ceil((n + 1) * (1.0 - alpha)))
    k = min(max(k, 1), n)
    ordered = np.sort(scores, kind="mergesort")
    return float(ordered[k - 1])

def prediction_sets_aps(probs: np.ndarray, q: float) -> np.ndarray:
    _, cumulative, _ = sorted_probabilities(probs)
    included_sorted = cumulative <= (q + 1e-12)
    order = np.argsort(-probs, axis=1, kind="stable")
    sets = np.zeros_like(probs, dtype=bool)
    sets[np.arange(len(probs))[:, None], order] = included_sorted
    return sets
```

7 Processing and Experiment Execution Procedure

7.1 Experiment Matrix Construction

In Section 12, `build_experiment_specs(seed)` constructs a list of 149 `ExperimentSpec` dataclass instances for each seed $s \in \{13, 42, 123, 456, 765\}$. Multiplying 149 specifications by the 3 conformal methods (`APS`, `TS+APS`, `RAPS`) yields 447 runs per seed and $447 \times 5 = 2{,}235$ total runs in `experiment_manifest.csv`. Table 2 summarizes the experimental groups generated per seed.

Table 2: Breakdown of the 149 `ExperimentSpec` instances generated per calibration seed by `build_experiment_specs(seed)`.

Group	Cal. Split	Test Split	Prompt Conditions	Grid	Specs/Seed	Total Runs
E1	clean (5k)	clean (10k)	P0, ensemble (2)	1 (none, 0)	2	30
E5	clean (5k)	clean (10k)	P1, P2, P3 (3)	1 (none, 0)	3	45
E2-E4	clean (5k)	corrupted (5k)	P0, ensemble (2)	$6 \times 3 = 18$	36	540
E6	clean (5k)	corrupted (5k)	P1, P2, P3 (3)	$6 \times 3 = 18$	54	810
E7	matched_shift (2k)	corrupted (5k)	P0 (1)	$6 \times 3 = 18$	18	270
E8	matched_shift (2k)	corrupted (5k)	P1, ensemble (2)	$6 \times 3 = 18$	36	540
Total					**149**	**2,235**

7.2 Resumable Execution Loop

In Section 14, `run_all_experiments()` first calls `load_checkpoint()` to read `results_checkpoint.csv` if it exists, validating that all 24 columns in `RESULT_COLUMNS` are present, non-null, and free of duplicate keys `(method, seed, calibration_condition, prompt_condition, visual_condition, corruption, severity)`. It then iterates over each seed $s \in$ `CALIBRATION_SEEDS`, calls `set_global_seed(s)`, iterates through `build_experiment_specs(s)` and `METHODS`, skips any key already in `done`, and otherwise executes `run_single_experiment(spec, method, s)` followed immediately by `append_checkpoint_row(row)`. After all 2,235 keys are completed, the checkpoint table is sorted deterministically (`kind="stable"`) and written to `results/results.csv`.

8 Configuration and Parameter Handling

Section 2 defines all protocol constants at module level and serializes a subset of them to `metadata/config.json`. Table 3 documents the configuration constants, their values, and their operational effects.

Table 3: Configuration constants defined in Section 2 and Section 4.

Constant Name	Value	Operational Effect
PRIMARY_SEED	42	Initial global RNG seed and base seed for bootstrap CI generation.
CALIBRATION_SEEDS	[13, 42, 123, 456, 765]	Seeds controlling clean calibration splits and matched-shift splits.

Continued on next page

Continued from previous page

Constant Name	Value	Operational Effect
ALPHA	0.10	Nominal miscoverage rate α passed to `conformal_quantile`.
TARGET_COVERAGE	0.90	Nominal coverage $1 - \alpha$ used in `coverage_gap` and plot reference lines.
CALIBRATION_SIZE	5000	Total sample count (500/class) for clean calibration splits.
SHIFT_TEST_SIZE	5000	Total sample count (500/class) for the fixed shifted test subset.
MATCHED_CALIBRATION_SIZE	2000	Total sample count (200/class) for matched-shift calibration splits.
MATCHED_SHIFT_TEST_SIZE	5000	Total sample count (500/class) for matched-shift evaluation splits.
SHIFT_SUBSET_SEED	20270317	RNG seed used to draw `shift_test_indices`.
MATCHED_BASE_SEED	914159	Base seed combined with each calibration seed via `stable_condition_seed`.
MODEL_NAME	"ViT-B/32"	Key in `CLIP_MODEL_REGISTRY` selecting the CLIP backbone.
NUM_CLASSES	10	Number of CIFAR-10 classes; used for stratification and matrix shape assertions.
PROMPT_ENSEMBLE	"ensemble_mean_logits"	Identifier triggering arithmetic-mean logit ensembling across all 4 prompts.
SEVERITIES	[1, 3, 5]	CIFAR-10-C severity levels evaluated in the experiment matrix.
METHODS	["APS", "TS+APS", "RAPS"]	Conformal methods evaluated for each `ExperimentSpec`.
RAPS_K_REG	5	Rank threshold k_{reg} beyond which RAPS applies a linear penalty.
RAPS_LAMBDA	0.01	Regularization weight λ per rank step above `RAPS_K_REG`.
ECE_BINS	15	Number of equal-width confidence bins in $[0, 1]$ for ECE calculation.
TEMPERATURE_MIN, MAX	0.05, 20.0	Lower and upper bounds on fitted scalar temperature $T(w)$.
TEMPERATURE_MAX_ITER	50	Maximum L-BFGS iterations in `fit_temperature`.
TEMPERATURE_TOL	1e-8	Gradient and parameter-change tolerance for L-BFGS in `fit_temperature`.
USE_GOOGLE_DRIVE	True	Selects `DRIVE_ROOT` (mounts Google Drive) vs. `LOCAL_ROOT`.
AUTO_DOWNLOAD_MISSING	True	Enables automatic network downloads for missing datasets and weights.
VERIFY_CIFAR10C_MD5	True	Enables MD5 verification of `CIFAR-10-C.tar`.
VERIFY_CIFAR10C_SHA256	True	Enables SHA-256 verification of `CIFAR-10-C.tar`.
INITIAL_BATCH_SIZE	128 (CUDA) / 32 (CPU)	Starting DataLoader batch size for `encode_image_embeddings`.

Continued on next page

Continued from previous page

Constant Name	Value	Operational Effect
`MIN_BATCH_SIZE`	`8`	Minimum batch size floor during CUDA OOM backoff.
`NUM_WORKERS`	`0`	DataLoader worker process count (main-process loading).

9 Evaluation, Aggregation, and Postprocessing Implementation

9.1 Per-Condition Evaluation Metrics

In Section 11, `evaluate_predictions(probs, pred_sets, labels)` validates that `probs` is a finite, row-normalized $(N, 10)$ float64 matrix and that `pred_sets` is an $(N, 10)$ boolean matrix $\widehat{C} \in \{0,1\}^{N\times 10}$, where $\widehat{C}_{i,j} = \mathbb{I}[j \in \widehat{\mathcal{C}}(x_i)]$. Let $y_i \in \{0,\dots,9\}$ be the true label, $\hat{y}_i = \arg\max_j \hat{\pi}_{i,j}$ the top-1 predicted class, $\hat{p}_i = \max_j \hat{\pi}_{i,j}$ the top-1 confidence, and $|\widehat{\mathcal{C}}_i| = \sum_{j=0}^{9} \widehat{C}_{i,j}$ the prediction-set cardinality for sample $i \in \{1,\dots,N\}$. The function computes nine scalar metrics:

$$\texttt{accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\hat{y}_i = y_i], \tag{15}$$

$$\texttt{coverage} = \frac{1}{N}\sum_{i=1}^{N} \widehat{C}_{i,y_i}, \tag{16}$$

$$\texttt{coverage_gap} = \big|\texttt{coverage} - (1-\alpha)\big|, \tag{17}$$

$$\texttt{average_set_size} = \frac{1}{N}\sum_{i=1}^{N} |\widehat{\mathcal{C}}_i|, \tag{18}$$

$$\texttt{singleton_rate} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\big[|\widehat{\mathcal{C}}_i| = 1\big], \tag{19}$$

$$\texttt{empty_set_rate} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\big[|\widehat{\mathcal{C}}_i| = 0\big], \tag{20}$$

$$\texttt{NLL} = -\frac{1}{N}\sum_{i=1}^{N} \log\big(\max(\hat{\pi}_{i,y_i}, 10^{-12})\big), \tag{21}$$

$$\texttt{entropy} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=0}^{9} \hat{\pi}_{i,j} \log\big(\max(\hat{\pi}_{i,j}, 10^{-12})\big). \tag{22}$$

For Expected Calibration Error (`expected_calibration_error`), the interval $[0,1]$ is partitioned into $B = 15$ equal-width bins $I_b = [e_{b-1}, e_b)$ for $b \in \{1,\dots,14\}$ and $I_{15} = [e_{14}, e_{15}]$ where $e_b = b/B$:

$$\texttt{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left| \frac{1}{|B_b|}\sum_{i\in B_b} \mathbb{I}[\hat{y}_i = y_i] - \frac{1}{|B_b|}\sum_{i\in B_b} \hat{p}_i \right|, \quad B_b = \{i : \hat{p}_i \in I_b\}. \tag{23}$$

9.2 Selective Top-1 Risk-Coverage Curves

In `risk_coverage_curve(probs, labels, n_points=101)`, samples are sorted in descending order of top-1 confidence $\hat{p}_i$ using a stable sort. For each fraction $\gamma \in$

`np.linspace(0.01, 1.0, 101)`, the function retains the top $k = \max(1, \lceil \gamma N \rceil)$ most confident samples and records the realized selection coverage k/N and selective top-1 classification risk:

$$\text{risk}(k) = 1 - \frac{1}{k} \sum_{m=1}^{k} \mathbb{I}[\hat{y}_{(m)} = y_{(m)}]. \tag{24}$$

9.3 Seed-Level Statistical Aggregation

In Section 15, `aggregate_results(results)` groups `results_df` by the 10 condition-identifying columns (`method`, `calibration_condition`, `prompt_condition`, `visual_condition`, `corruption`, `severity`, `test_size`, `calibration_size`, `alpha`, `target_coverage`), yielding $2{,}235/5 = 447$ groups of 5 seeds each. After verifying that each group contains no duplicate seed values, it computes seven summary statistics for each of the 9 metrics in `metric_cols`:

- `<metric>_mean`: sample mean across the $m = 5$ seeds.
- `<metric>_std`: sample standard deviation with Bessel's correction (`ddof=1`).
- `<metric>_median`, `<metric>_min`, `<metric>_max`: median, minimum, and maximum across the 5 seeds.
- `<metric>_ci95_low` and `<metric>_ci95_high`: 2.5th and 97.5th percentiles of 4,000 bootstrap means computed by `bootstrap_seed_ci(values, ci_seed, n_boot=4000)`, where `ci_seed` is derived deterministically from `(PRIMARY_SEED, "bootstrap_ci", *keys, metric)` via `_bootstrap_condition_seed`.

10 Output Generation and File Handling

Sections 16, 17, 19, 20, and 21 generate all persistent tables, plots, documentation files, and archives:

- **Figures (Section 16 / Cell 32):** All figures are rendered at 300 DPI (`bbox_inches="tight"`) into `results/figures/`:
  1. `figure_1_coverage_vs_severity.png`: Line plot of mean empirical coverage vs. severity $s \in \{1, 3, 5\}$ averaged over the six corruptions for `APS`, `TS+APS`, and `RAPS` under clean calibration and `P0_canonical`.
  2. `figure_2_set_size_vs_severity.png`: Line plot of mean prediction-set size vs. severity $s \in \{1, 3, 5\}$ for the same slice.
  3. `figure_3_coverage_shift_conditions.png`: Bar chart comparing mean `APS` coverage across four condition groups (`same prompt / clean`, `prompt shift / clean`, `visual shift / same prompt` at severity 3, and `combined shift` at severity 3).
  4. `figure_4_risk_coverage.png`: Selective top-1 risk-coverage curves for seed 13 across `APS`, `TS+APS`, and `RAPS` on clean test data (`P0_canonical`) and on `motion_blur` severity 5 (`P1_paraphrase`).
  5. `figure_5_coverage_gap_heatmap.png`: Viridis heatmap of mean absolute coverage gap across the 6×3 corruption-by-severity grid for `APS` under clean calibration and `P0_canonical`.

6. `figure_6_clean_vs_matched_shift.png`: Grouped bar chart comparing clean calibration vs. matched-shift calibration across all six corruptions at severity 3 for `APS` and `P0_canonical`.

- **Programmatic Tables (Section 17 / Cell 34):** Filters `aggregate_df` to create `main_comparison_table.csv` (57 rows covering clean `P0`, corrupted severity-5 `P1` under clean calibration, and corrupted severity-5 `P0` and `P1` under matched-shift calibration across all three methods), renders matching `.html` and `.png` versions, and exports `ablation_summary_APS.csv`.
- **Packaging (Section 21 / Cell 42):** Copies all generated CSVs, PNGs, HTML tables, split `.npz` files, metadata JSONs, and Markdown files into `final_artifacts/`, computes file sizes and SHA-256 checksums into `final_artifacts/artifact_manifest.csv`, compresses the directory into `prompt_shift_conformal_vlm_final_artifacts.zip`, and triggers browser downloads when running inside Google Colab.

11 Error Handling, Reproducibility, and Execution Considerations

11.1 Randomness Control and Determinism

The function `set_global_seed(seed)` seeds `random`, `numpy.random`, `torch.manual_seed`, and `torch.cuda.manual_seed_all`, enables `torch.use_deterministic_algorithms(True, warn_only=True)`, and sets `torch.backends.cudnn.deterministic = True` and `benchmark = False`. All split-construction and bootstrap routines instantiate isolated `np.random.default_rng` generators seeded with explicit integer constants rather than relying on global NumPy RNG state. Furthermore, all array sorting operations that may encounter ties (`conformal_quantile`, `sorted_probabilities`, `prediction_sets_aps`, `prediction_sets_raps`, `risk_coverage_curve`, and DataFrame `sort_values`) specify `kind="mergesort"` or `kind="stable"`.

11.2 Exception Handling and Forensic Auditing

Rather than silently ignoring corrupted caches or failed downloads, the code validates shapes, dtypes, norms, sample IDs, label arrays, and file hashes before reuse. If a cached `.npz` or `.json` file fails validation, the code issues a `warnings.warn` diagnostic and recomputes the artifact. If a dataset archive or model checkpoint fails MD5/SHA-256 verification, it unlinks the invalid file and raises a `RuntimeError`. In Section 18, `run_forensic_audit(results_df)` programmatically verifies seven invariants before packaging:

1. Prompt condition names match `set(PROMPTS.keys()) | {PROMPT_ENSEMBLE}`.
2. Clean calibration (train partition) and clean test (test partition) are disjoint by dataset partition.
3. Every seed's matched-shift calibration and test index arrays have zero intersection (`overlap == []`).
4. Sample logits have shape `(128, 10)` and resulting probabilities pass `finite_and_normalized_probabilities`.
5. `results_df` contains all 2,235 manifest rows with zero duplicate keys and zero `NaN` entries.

6. Nominal `alpha` (0.10), `target_coverage` (0.90), method names, and calibration sizes (5,000 or 2,000) match protocol constants across all rows.

7. Core cache files (`text_features.npz` and `clean_test_all.npz`) exist on disk.

12 Implementation Limitations

The following limitations follow directly from the supplied source code:

1. **Empty prediction sets under deterministic APS/RAPS:** Because `prediction_sets_aps` and `prediction_sets_raps` test `cumulative <= (q + 1e-12)` without randomized tie-breaking or forced top-1 inclusion, any test sample whose top-1 probability $\pi_{(1)}(x)$ exceeds $\hat{q} + 10^{-12}$ receives an empty prediction set ($\widehat{\mathcal{C}}(x) = \emptyset$).

2. **Single-model and fixed-dataset scope:** `CLIP_MODEL_REGISTRY` contains only `"ViT-B/32"`, and dataset loaders are hardcoded to 10-class CIFAR-10 and $(50000, 32, 32, 3)$ CIFAR-10-C arrays.

3. **Redundant clean calibration state files:** `calibration_state_path` includes `corruption` and `severity` in the JSON filename even when `calibration_condition == "clean"`, producing 19 identical JSON files per `(seed, prompt_condition, method)` tuple.

4. **Unaggregated columns in `aggregate_results`:** `empty_set_rate` and `conformal_quantile` are saved in `results.csv` but are not included in `metric_cols` inside `aggregate_results`.

5. **Colab and external network assumptions:** Initial execution when caches are absent requires outbound HTTPS access to Zenodo (2.92 GB), OpenAI Azure CDN (338 MB), and the torchvision CIFAR-10 mirror.

13 Summary of the Implementation

The reviewed project implements a complete, self-verifying pipeline for evaluating zero-shot CLIP `ViT-B/32` conformal prediction sets under prompt and visual distribution shifts. By decoupling image encoding from prompt-specific logit computation and persisting intermediate splits, embeddings, logits, calibration states, and incremental result rows, the architecture avoids redundant neural network forward passes across prompt templates and supports interruption-tolerant execution. During the code audit, duplicate helper definitions were consolidated, a split-label slicing bug in the dead `save_indices` definition was corrected, a shifted-test embedding/logit cache collision between clean-calibrated and matched-shift evaluations was resolved, a pre-filtering bug that excluded clean test conditions from Figure 3 was fixed, a Pillow `DeprecationWarning` was removed, and formatting was standardized across the script and notebook while preserving full behavioral and schema compatibility.

A Function and Class Interface Reference

Table 4 documents the classes and functions in the reviewed codebase.

Table 4: Summary of classes and functions in `prompt_shift_conformal_vlm_persistent.py`.

Name	Arguments / Signature	Return Value & Role
`set_global_seed`	`(seed: int) -> None`	Seeds Python, NumPy, and PyTorch RNGs; enables deterministic algorithms.
`environment_metadata`	`() -> Dict[str, Any]`	Returns runtime software, hardware, and git revision dictionary.
`sha256_file`, `md5_file`	`(path: Path, chunk_size: int) -> str`	Stream file in 8 MiB chunks and return hex digest.
`sanitize_name`	`(value: str) -> str`	Normalizes string to alphanumeric, `-`, and `_` characters.
`atomic_write_json`	`(path: Path, payload: Mapping) -> None`	Writes JSON to `.tmp` file and renames atomically.
`stable_condition_seed`	`(base_seed: int, condition: int) -> int`	Deterministic bitwise XOR seed derivation for matched-shift splits.
`_bootstrap_condition_seed`	`(base_seed: int, *parts: object) -> int`	Deterministic variadic SHA-256 seed derivation for bootstrap CIs.
`stratified_indices`	`(labels: Sequence[int], size: int, seed: int) -> np.ndarray`	Returns shuffled class-balanced `int64` index array of length `size`.
`save_indices`	`(path: Path, indices: np.ndarray, labels: np.ndarray, metadata: Mapping) -> None`	Saves `indices`, `labels[indices]`, and JSON `metadata` to `.npz`.
`load_valid_indices`	`(path: Path, expected_size: int, reference_labels: Optional[np.ndarray]) -> Optional[np.ndarray]`	Validates and returns cached split indices, or `None` if invalid.
`finite_and_normalized_probabilities`	`(probs: np.ndarray, atol: float) -> None`	Asserts shape `(N, 10)`, finiteness, bounds in $[0, 1]$, and unit row sums.
`cache_npz_valid`	`(path: Path, required_arrays: Sequence[str]) -> bool`	Checks whether `path` is a valid `.npz` containing all required keys.
`_format_bytes`, `_format_seconds`	`(num_bytes) -> str`, `(seconds) -> str`	Human-readable byte and duration formatters for download logs.
`download_with_progress`	`(url, destination, expected_size, chunk_size, max_retries) -> Path`	Resumable HTTP downloader with `Range` support and `tqdm` bar.
`_load_or_download_cifar10_split`, `ensure_cifar10`	`(train: bool) -> CIFAR10`, `() -> Tuple[CIFAR10, CIFAR10]`	Load and validate official CIFAR-10 train and test datasets.
`_verify_cifar10c_archive`, `ensure_cifar10c_archive`	`(path: Path) -> None`, `() -> Path`	Ensure `CIFAR-10-C.tar` exists and passes MD5/SHA-256 checks.
`_find_tar_member`, `_valid_extracted_cifar10c_file`, `_all_required_cifar10c_files_present`, `extract_selected_cifar10c_members`	Archive member lookup, validation, and selective extraction helpers	Return `Dict[str, Path]` mapping corruption names and `"labels"` to validated `.npy` paths.

Continued on next page

Continued from previous page

Name	Arguments / Signature	Return Value & Role
`CIFAR10CSubset`	`Dataset` subclass	Memmap-backed CIFAR-10-C dataset for a single corruption and severity.
`verify_cifar10c_alignment`	`(extracted: Mapping[str, Path], clean_test: CIFAR10) -> None`	Verifies label block equality and corruption array shapes/dtypes.
`ensure_clip_weights`	`() -> Path`	Verifies or downloads `ViT-B-32.pt` and checks SHA-256.
`build_text_features`, `load_or_build_text_features`	`() -> Dict[str, np.ndarray]`	Compute or load L2-normalized $(10, 512)$ text embeddings per prompt.
`_make_loader`, `CleanCIFARSubset`, `encode_image_embeddings`, `get_shift_embeddings`	Image feature extraction and caching routines	Compute or load L2-normalized $(N, 512)$ CLIP image embeddings with CUDA OOM backoff.
`logits_from_embeddings`, `logits_cache_path`, `logits_cached`	Logit computation and caching routines	Compute and cache $(N, 10)$ scaled cosine-similarity or ensemble logits.
`probability_from_logits`	`(logits: np.ndarray, temperature: float) -> np.ndarray`	Computes temperature-scaled softmax probabilities in `float64`.
`_inv_sigmoid`, `fit_temperature`	`(logits: np.ndarray, labels: np.ndarray) -> float`	Fits constrained scalar temperature $T \in [0.05, 20.0]$ via L-BFGS.
`conformal_quantile`, `sorted_probabilities`, `aps_calibration_scores`, `raps_calibration_scores`, `prediction_sets_aps`, `prediction_sets_raps`, `fit_conformal_state`, `build_prediction_sets`	Conformal scoring, quantile, and set-construction routines	Compute finite-sample quantile $\hat{q}$ and boolean prediction-set matrices for `APS`, `TS+APS`, and `RAPS`.
`accuracy`, `coverage`, `expected_calibration_error`, `nll`, `entropy`, `evaluate_predictions`, `risk_coverage_curve`	Evaluation metric routines	Compute per-run accuracy, coverage, set-size, calibration, and selective risk metrics.
`ExperimentSpec`, `result_key`, `build_experiment_specs`	Experiment matrix specification helpers	Materialize the 149 `ExperimentSpec` instances per seed.
`calibration_logits_for`, `calibration_state_path`, `fit_or_load_calibration_state`, `load_checkpoint`, `append_checkpoint_row`, `load_or_build_test_embeddings`, `run_single_experiment`, `run_all_experiments`	Experiment execution and checkpointing routines	Execute all 2,235 runs with per-row CSV checkpointing and calibration-state caching.
`bootstrap_seed_ci`, `aggregate_results`	Statistical aggregation routines	Aggregate metrics across the 5 calibration seeds with 4,000-sample percentile bootstrap CIs.

Continued on next page

Continued from previous page

Name	Arguments / Signature	Return Value & Role
`savefig`, `pretty_corruption`, `compute_risk_coverage_for_condition`	Plotting helpers	Generate and save Figures 1–6 and `risk_coverage.csv`.
`run_forensic_audit`	`(results: pd.DataFrame) -> Dict[str, object]`	Verifies split disjointness, probability normalization, and result completeness.

B Notebook-to-Script Cell Mapping

Table 5 maps all 43 cells of `prompt_shift_conformal_vlm_persistent.ipynb` to the sections of `prompt_shift_conformal_vlm_persistent.py`.

Table 5: Cell-by-cell mapping of `prompt_shift_conformal_vlm_persistent.ipynb`.

Cell	Type	ID	Section / Responsibility
0	Code	`5c0a6ce5`	UTF-8 encoding declaration (`# -*- coding: utf-8 -*-`).
1	Markdown	`f61a7d1d`	Title, scientific protocol overview, and Section 1 introduction.
2	Code	`820ebe53`	Section 1: `pip install` and import verification for `clip`, `ftfy`, `regex`, `tqdm`.
3	Markdown	`75669827`	Section 2 introduction (central configuration and reproducibility policy).
4	Code	`6e774fc6`	Section 2: Imports, protocol constants, Drive mount, directory setup, `set_global_seed`, `environment_metadata`.
5	Markdown	`410211fa`	Section 3 introduction (utility functions and integrity checks).
6	Code	`11beba7b`	Section 3: Hashing, atomic JSON, seed derivation, stratified sampling, split I/O, probability/cache validation.
7	Markdown	`3fff875b`	Section 4 introduction (CIFAR-10 and CIFAR-10-C acquisition and splits).
8	Code	`38f5c151`	Section 4: Resumable downloader, archive verification/extraction, `CIFAR10CSubset`, alignment check, split generation.
9	Markdown	`102297cf`	Section 5 introduction (prompt definitions and frozen CLIP loading).
10	Code	`f502da1d`	Section 5: `ensure_clip_weights`, frozen CLIP loading, `build_text_features`, `TEXT_FEATURES`.
11	Markdown	`d58b854f`	Section 6 introduction (image inference and embedding caches).
12	Code	`b7f2e62a`	Section 6: `encode_image_embeddings`, `CleanCIFARSubset`, clean test and calibration embedding extraction.
13	Markdown	`a63a5c00`	Section 7 introduction (CIFAR-10-C lazy loading and shift embedding cache).
14	Code	`771a82d4`	Section 7: `SHIFT_EMBED_ROOT` and `get_shift_embeddings`.
15	Markdown	`cd8cd5f0`	Section 8 introduction (zero-shot logits and prompt-ensemble construction).
16	Code	`92c4a6d7`	Section 8: `logits_from_embeddings`, `logits_cached`, `probability_from_logits`.
17	Markdown	`273bb433`	Section 9 introduction (post-hoc scalar temperature scaling).
18	Code	`34c78323`	Section 9: `_inv_sigmoid` and `fit_temperature` (L-BFGS optimizer).

Cell	Type	ID	Section / Responsibility
19	Markdown	e310fbeb	Section 10 introduction (deterministic APS and RAPS).
20	Code	b5e126cf	Section 10: conformal_quantile, sorted_probabilities, APS/RAPS scores and prediction sets.
21	Markdown	e1468320	Section 11 introduction (evaluation metrics).
22	Code	28f32dcc	Section 11: accuracy, coverage, expected_calibration_error, nll, entropy, evaluate_predictions, risk_coverage_curve.
23	Markdown	29316ac1	Section 12 introduction (experiment matrix E1–E8).
24	Code	37dd244a	Section 12: ExperimentSpec, result_key, build_experiment_specs, experiment_manifest.csv.
25	Markdown	a3d5414e	Section 13 introduction (lightweight end-to-end sanity experiment).
26	Code	2e435b29	Section 13: 128-calibration / 256-test sanity verification of TS+APS.
27	Markdown	76372b23	Section 14 introduction (calibration-state caching and resumable execution).
28	Code	3090580f	Section 14: calibration_logits_for, fit_or_load_calibration_state, checkpoint I/O, run_all_experiments.
29	Markdown	800c1dae	Section 15 introduction (statistical aggregation).
30	Code	466c59f1	Section 15: bootstrap_seed_ci, aggregate_results, results_aggregated_across_seeds.csv.
31	Markdown	c7fd5e67	Section 16 introduction (figure generation).
32	Code	59d739cb	Section 16: Generation of Figures 1–6 and risk_coverage.csv.
33	Markdown	00de748e	Section 17 introduction (programmatic result tables).
34	Code	eda74b9b	Section 17: Generation of main_comparison_table.{csv,html,png} and ablation_summary_APS.csv.
35	Markdown	21e1500e	Section 18 introduction (forensic sanity and leakage checks).
36	Code	6489eef8	Section 18: run_forensic_audit and metadata/forensic_audit.json.
37	Markdown	272b8476	Section 19 introduction (protocol, limitations, requirements, and README files).
38	Code	a0b59cb9	Section 19: Writes README.md, EXPERIMENT_PROTOCOL.md, LIMITATIONS.md, requirements.txt under ROOT.
39	Markdown	bb69633d	Section 20 introduction (pipeline diagram).
40	Code	727d8ddf	Section 20: Draws and saves assets/pipeline_diagram.png.
41	Markdown	2cd29d2a	Section 21 introduction (final artifact collection and Colab downloads).
42	Code	7ab766d7	Section 21: Writes persistence_status.json, populates final_artifacts/, writes artifact_manifest.csv, builds ZIP archive.

C Summary of Code Cleanup and Audit Findings

Table 6 summarizes the substantive modifications recorded in code_changes.py. None of these modifications alter experimental hyperparameters, random seeds, dataset splits, model weights, or metric definitions.

Table 6: Summary of substantive code modifications recorded in `code_changes.py`.

Location	**Modification**	**Effect on Behavior**
Header & Cell 1	Removed duplicate Colab header docstrings and stale chat-session disclaimer.	Documentation clarity only.
Cells 2, 4, 8, 10, 42	Removed unused imports (`os`, `copy`, `csv`, `asdict`, `Iterable`, `Subset`) and consolidated duplicate imports into Cell 4.	Eliminates redundant import statements; all symbols remain bound.
Cells 6, 8, 30	Consolidated duplicate helpers into Cell 6, fixed label slicing in `save_indices`, and separated `stable_condition_seed` (XOR) from `_bootstrap_condition_seed` (SHA-256).	Eliminates function shadowing and fixes Cell 6's dead `save_indices` bug while preserving exact runtime seeds.
Cell 8 (`CIFAR10CSubset`)	Removed deprecated `mode="RGB"` parameter from `Image.fromarray(image_array)`.	Eliminates Pillow `DeprecationWarning`; RGB inference is identical for $(32, 32, 3)$ `uint8` arrays.
Cell 22 (`evaluate_predictions`)	Checked `pred_sets.dtype != np.bool_` directly and computed `coverage(pred_sets, labels)` once per call.	Avoids redundant array indexing; identical metric values.
Cell 28 (`load_or_build_test_embeddings`, `run_single_experiment`)	Used seed-specific cache tags (`matched_test_seed_<seed>` and `matched_<corr>_s<sev>_seed_<seed>_test`) for `matched_shift` test sets.	Prevents cache-file thrashing between fixed `shift_test` and seed-specific `matched_shift` test sets; identical numerical outputs.
Cell 32 (Figure 3 `base3`)	Included `visual_condition == "clean"` rows in `base3` alongside severity-3 corrupted rows.	Populates all four condition groups defined in `np.select` for Figure 3.
All Sections	Removed ornamental comment banners (`# ===...===`), normalized 35-column line wrapping to PEP 8 widths, and normalized non-ASCII punctuation.	Improves code readability and maintainability with zero runtime impact.